

KOTLIN OOP REFLECTION CHALLENGE

GROUP MEMBER

172638

182654

189797

190068

193001

Kotlin OOP Reflection Challenge

Project: Real Estate and Property Management App

1. Encapsulation

One object in our system is a property. Information such as the property ID, owner, price, availability status, and property details should be kept private. Other parts of the system should not be able to change this information directly because unauthorized changes could cause incorrect property information or affect transactions. The information should only be changed through authorized operations in the system.

2. Abstraction

Our system allows a user to search for and view available properties without knowing how the system works internally. The user only needs to enter their search requirements, such as location, property type, or price range. They do not need to know how the application searches the database, filters the properties, or retrieves the results.

3. Inheritance

Two objects that have something in common are Tenant and Property Owner. Both are users of the system and can have common information such as a user ID, name, email, phone number, and login details.

However, they have different responsibilities. A Tenant can search for properties, make rental requests, and manage their rental information, while a Property Owner can add properties, update property details, and manage tenants.

4. Polymorphism

One action that can be performed in different ways is making a payment. Different payment methods can process the payment differently, for example:

- Mobile Money – processes the payment through a mobile money service.
- Bank Transfer – processes the payment through a bank account.
- Card – processes the payment through a debit or credit card.

Although the general action is making a payment, the way the payment is processed depends on the payment method.

5. Reflection

As a group, we think Encapsulation would be the easiest OOP principle to apply to our project because our application has clear objects such as properties, tenants, owners, and payments. It is easy to identify information that should be protected and controlled.

We think Polymorphism would be the most difficult because different users and services can perform similar actions in different ways. It would require careful planning to make sure that the different behaviours are organized properly without making the system unnecessarily complicated.